

Relatório Trabalho 1 OrgB

Arthur Padilha, Johann Hardt, Tobias Marion, Vítor Feijó

1 Implementação do JAL

A implementação da instrução JAL foi feita ao incluir o caminho de dados para “ $rd = PC+4$, $PC+=imm$ ” e/ou alterando o bloco de controle, conforme o necessário.

1.1 Monociclo

No monociclo, foi usado um comparador com o valor do opcode para gerar o sinal “Jump and Link”, que serve de seletor para os multiplexadores de dados de escrita PC e rd. Também foi alterado o bloco de controle (ROM), para gerar os sinais de controle adequados para o opcode da instrução JAL.

1.2 Multiciclo

No multiciclo, os caminhos de dados foram reaproveitados; o incremento de PC ($PC += 4$) já estava sendo realizado na ALU, cuja saída já possuía caminho para PC e rd. Portanto, foi ajustada somente a FSM do bloco de controle. Dois novos estados foram adicionados: O primeiro segue do segundo estado (decode), somando na ALU $PC+4$ enquanto é escrito no PC a saída anterior, “ $PC += imm$ ”. O segundo estado escreve a saída anterior da ALU em rd, completando a operação JAL.

1.3 Pipeline

No pipeline, similarmente a como foi feito no monociclo, foi usado um comparador para gerar o sinal “JAL”. O sinal é propagado pelos estágios do pipeline via registradores, e rege a escrita de “ $PC += imm$ ” no PC (em “OR” com a condição branch), assim como “ $rd = PC+4$ ” no último estágio.

2 Implementação do BLT

A estratégia geral para a implementação da instrução BLT (Branch if Less Than) consistiu em utilizar a Unidade Lógica e Aritmética (ALU) para realizar uma subtração ($rs1 - rs2$) e verificar o bit de sinal do resultado. Como a arquitetura RISC-V utiliza complemento de dois, um resultado negativo — indicando que

$rs1 < rs2$ — possui obrigatoriamente o bit mais significativo (bit 31) em nível lógico alto. Para capturar essa condição, adicionou-se um *splitter* na saída de 32 bits da ALU para isolar o bit 31, criando a flag customizada **isNegative**. Um multiplexador foi então inserido para selecionar entre a flag **Zero** (utilizada pelo BEQ) e a nova flag **isNegative**, utilizando o bit 2 do campo **funct3** da instrução como sinal de controle para definir qual condição de salto deve ser enviada para a lógica de desvio.

2.1 Organização Monociclo

No processador monociclo, a implementação foi realizada integrando o multiplexador de seleção de flag diretamente ao caminho de dados entre a ALU e a porta AND que controla o sinal de *branch*. Como a instrução é executada em um único ciclo, o bit 2 do **funct3** extraído da memória de instruções roteia instantaneamente a flag correta para a lógica de controle, permitindo que o PC seja atualizado com o endereço de destino do salto no exato momento em que a ALU estabiliza o resultado da operação aritmética.

2.2 Organização Multiciclo

Na organização multiciclo, a implementação exigiu a sincronização com a máquina de estados, onde o campo **funct3** deve ser lido a partir do *Instruction Register* (IR). A decisão do salto ocorre no terceiro ciclo (Execução), onde a ALU realiza a subtração e a flag **isNegative** é selecionada pelo multiplexador. Caso a condição seja verdadeira, a porta AND ativa o sinal **PCWriteCond**, forçando o PC a carregar o endereço de destino que foi previamente calculado no ciclo de decodificação e armazenado no registrador intermediário **ALUOut**.

2.3 Organização Pipeline

No pipeline, a estratégia foi adaptada para respeitar a divisão de estágios e evitar conflitos de temporização. O multiplexador de seleção e a geração da flag **isNegative** foram posicionados no estágio **EX** (Execução), logo após a ALU. O resultado dessa comparação (um único bit indicando se a condição de "menor que" foi atendida) é armazenado no registrador de barreira EX/MEM e propagado para o estágio **MEM** (Memória). Somente neste estágio a porta AND de desvio é acionada, utilizando o bit propagado para validar o salto e atualizar o multiplexador do PC no início do pipeline.

3 Instrução LB (*Load Byte*)

A instrução LB possui opcode 0000011 e **funct3** = 000. Os sinais de controle são idênticos ao LW, portanto a unidade de controle não precisa ser modificada. A única diferença é que LB lê apenas os 8 bits menos significativos da memória e realiza extensão de sinal para 32 bits antes de escrever no registrador destino.

Componente adicionado: SignExtend8

Subcircuito criado no Logisim com entrada de 32 bits e saída de 32 bits. Internamente, os bits 7–0 da entrada são extraídos via Splitter. O bit 7 é então replicado nas posições 8 a 31 da saída (extensão de sinal), e os bits 7–0 originais são mantidos. Implementado com dois Splitters: um para separar os bits 7–0 do restante e outro para montar os 32 bits de saída com o sinal estendido.

Monociclo

1. Adicionar Splitter na saída da Instruction Memory para extrair **funct3** (bits 14–12).
2. Conectar a saída de 32 bits da Data Memory ao subcircuito SignExtend8.
3. Adicionar porta NOR de 3 entradas recebendo **funct3**. Saída = 1 quando **funct3** = 000 (LB).
4. Adicionar MUX entre a Data Memory e o MUX MemToReg:
 - Entrada 0: saída de 32 bits da Data Memory (LW)
 - Entrada 1: saída do SignExtend8 (LB)
 - Select: saída da porta NOR

Multiciclo

1. Adicionar Splitter no IR para extrair **funct3** (bits 14–12).
2. Conectar a saída de 32 bits do MDR ao subcircuito SignExtend8.
3. Adicionar porta NOR de 3 entradas recebendo **funct3**.
4. Adicionar MUX entre o MDR e o MUX MemToReg:
 - Entrada 0: saída de 32 bits do MDR (LW)
 - Entrada 1: saída do SignExtend8 (LB)
 - Select: saída da porta NOR

Pipeline

1. Propagar **funct3** (bits 14–12 da instrução) pelos registradores de pipeline IF/ID → ID/EX → EX/MEM → MEM/WB.
2. Conectar a saída de 32 bits da Data Memory ao subcircuito SignExtend8.
3. Adicionar porta NOR de 3 entradas recebendo o **funct3** propagado até MEM/WB.
4. Adicionar MUX entre a Data Memory e o registrador MEM/WB:

- Entrada 0: saída de 32 bits da Data Memory (LW)
- Entrada 1: saída do SignExtend8 (LB)
- Select: saída da porta NOR

4 Implementação da Instrução SRA

A implementação da instrução SRA (Shift Right Arithmetic) exigiu modificações em somente dois módulos principais, independente do tipo do processador(monociclo, multiciclo ou pipeline):

4.1 Unidade de Controle da ALU (ALUControl)

Foi adicionada a lógica para decodificar uma nova combinação de **funct3** e **funct7**. A nova operação é selecionada conforme a regra:

- **funct3:** 0x5 (101)
- **funct7:** 0x20 (0100000)
- **Saída(AluOp):** 0x5 (101)

4.2 Unidade Lógica e Aritmética (ALU)

Na ALU, implementou-se o suporte ao deslocamento aritmético. Utilizou-se o componente nativo de *shifter* do Logisim, configurado para o modo aritmético quando o seletor da ALU recebe o código 101.